Multilayer Perceptron

Dinanath Padhya*, Krishna Acharya†
*THA078BEI010 †THA078BEI020
Department of Electronics, Communication and Information Engineering
Thapathali Campus, IOE, Tribhuvan University, Nepal

***Abstract*—This experiment aims to implement and evaluate a Multi-Layer Perceptron (MLP) neural network model using PyTorch for binary classification on a synthetically generated dataset. The dataset was created using `sklearn`'s `make_classification`, comprising 5000 samples and 10 features, of which 3 are informative. A custom PyTorch `Dataset` class was utilized to efficiently handle data loading. The MLP model consists of three fully connected layers with LeakyReLU and Sigmoid activations, and it was trained using Stochastic Gradient Descent (SGD) and CrossEntropyLoss as the loss function. Evaluation was performed using accuracy metrics and loss curves. The model achieved an accuracy of approximately 89%, indicating that even simple neural network architectures can provide effective performance when trained properly. The study highlights the importance of activation functions, optimizers, and training strategies in achieving convergence and generalization. Results are visualized through training and validation loss plots, reinforcing the model's learning behavior across epochs.**

***Index Terms*—**

I. Introduction

A. Background

Artificial Neural Networks (ANNs) have emerged as powerful tools for solving a wide range of machine learning problems, including classification, regression, and pattern recognition. Among various types of ANNs, the Multi-Layer Perceptron (MLP) [1] stands as a foundational architecture that facilitates learning through feedforward layers and backpropagation. MLPs are capable of modeling complex, non-linear relationships in data, making them suitable for binary as well as multi-class classification problems.

B. Literature Review

Recent advances in neural network training, especially the use of diverse activation functions (like ReLU, LeakyReLU, Sigmoid), optimizers (such as SGD, Adam), and robust datasets, have significantly improved the learning capabilities of MLPs. Studies have shown that even shallow networks, when properly tuned, can perform on par with deeper architectures for simple tasks [2]. PyTorch [3] has become a widely adopted framework in academia and industry for developing and testing such models due to its flexibility and GPU acceleration.

C. Objective

The objective of this lab experiment is to design and implement a basic MLP classifier using PyTorch on a synthetic dataset. Key goals include:

- Generating and preprocessing a binary classification dataset.
- Designing an MLP model using linear layers and activation functions.
- Training the model using the SGD optimizer and evaluating performance using test accuracy and loss curves.
- Visualizing the learning behavior of the model across epochs.

II. Methodology

A. Implementation Environment

III. Theoretical Background

A. Activation Functions

Activation functions are mathematical functions applied to the output of each neuron in a neural network layer. Their primary purpose is to introduce non-linearity into the model, enabling neural networks to approximate complex, non-linear mappings between inputs and outputs. Without activation functions, a neural network, regardless of its depth, would behave as a single linear transformation, severely limiting its representational power.

1) Sigmoid: The sigmoid function is a smooth, S-shaped curve that maps any real-valued input to the range $(0, c)$, making it especially suitable for binary classification tasks. The general form of the sigmoid function with parameters a, b, and c is:

$$\sigma(x) = \frac{c}{a + be^{-x}} \tag{1}$$

where a, b, and c are constants that control the curve's shape:

- **Parameter** c sets the upper asymptote (maximum value) of the function.
- **Parameter** a affects the lower asymptote; when $a = 1$, the function ranges from 0 to c.
- **Parameter** b controls the steepness and direction; increasing b makes the curve steeper, while negative b reflects it horizontally.

The standard sigmoid, with $a = 1$, $b = 1$, $c = 1$, produces the classic S-curve from 0 to 1, crossing 0.5 at $x = 0$ (see Figure 1).

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \tag{2}$$

The sigmoid function is widely used in neural networks due to its differentiability and well-defined gradient, which is essential for backpropagation:

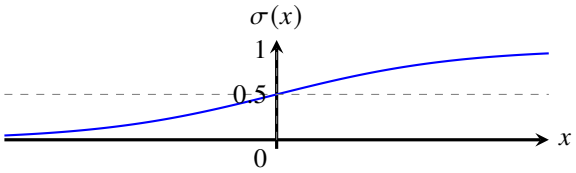


Fig. 1. Sigmoid Activation Function

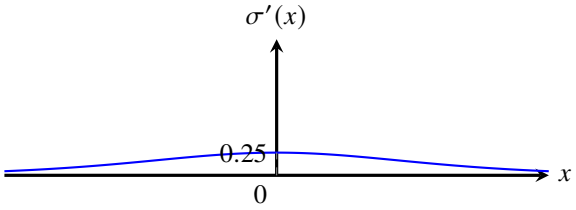


Fig. 2. Derivative of Sigmoid Activation Function: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

2) Tanh: Tanh outputs values in $(-1, 1)$, centering the data and often leading to faster convergence than sigmoid:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{3}$$

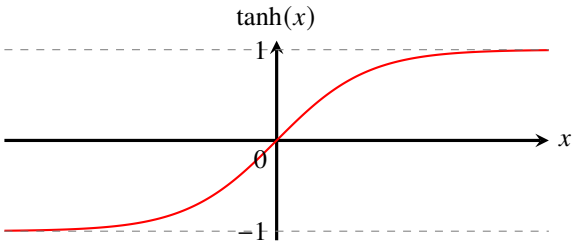


Fig. 3. Tanh Activation Function

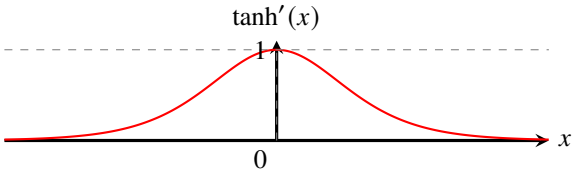


Fig. 4. Derivative of Tanh Activation Function: $\tanh'(x) = 1 - \tanh^2(x)$

Its derivative is:

$$\tanh'(x) = 1 - \tanh^2(x) \tag{4}$$

Like sigmoid, it can also suffer from vanishing gradients.

3) ReLU (Rectified Linear Unit): ReLU is widely used due to its simplicity and effectiveness:

$$f(x) = \max(0, x) \tag{5}$$

It outputs zero for negative inputs and the input itself for positive values. Its derivative is:

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

ReLU helps mitigate the vanishing gradient problem but can lead to dead neurons if too many outputs become zero.

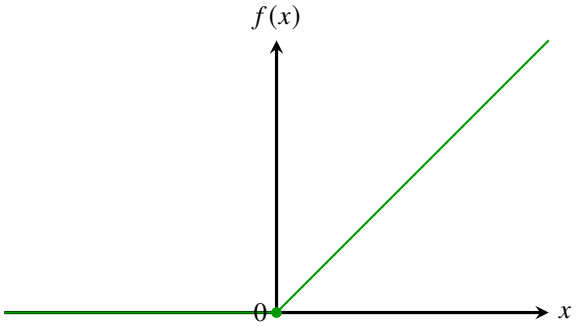


Fig. 5. ReLU Activation Function: $f(x) = \max(0, x)$

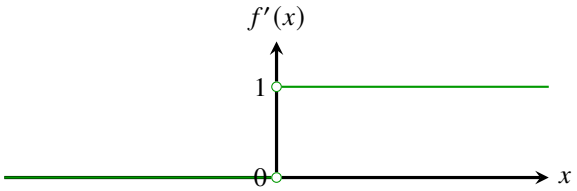


Fig. 6. Derivative of ReLU Activation Function

4) Leaky ReLU: This variant allows a small, non-zero gradient when the unit is not active:

$$f(x) = \max(\alpha x, x) \tag{7}$$

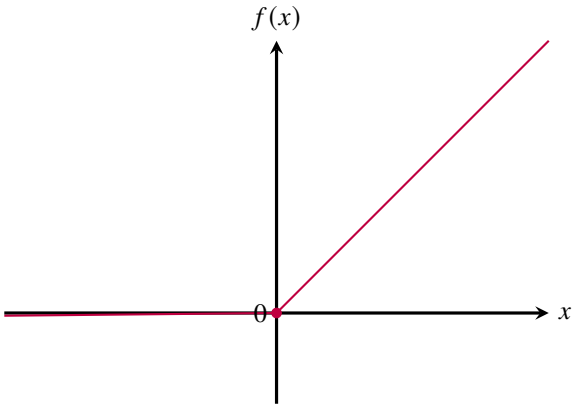


Fig. 7. Leaky ReLU Activation Function: $f(x) = \max(\alpha x, x)$ with $\alpha = 0.01$

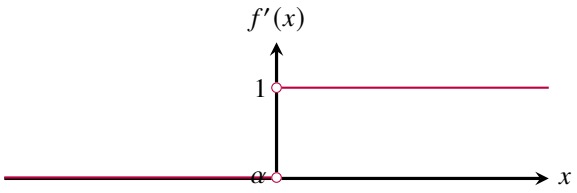


Fig. 8. Derivative of Leaky ReLU Activation Function: $f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{otherwise} \end{cases}$

Where α is a small constant (e.g., 0.01). This helps address the dead neuron issue.

5) Softmax: Used in the output layer for multi-class classification:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{8}$$

Softmax converts logits into probabilities that sum to 1. Its derivative is more complex, involving the Jacobian matrix.

Activation functions are chosen based on the problem type and network architecture. Their derivatives are essential for gradient-based optimization, as they determine how errors are propagated backward through the network during training.

B. Pseudo-Random Number Generators (PRNGs)

Pseudo-random number generators (PRNGs) are deterministic algorithms that produce sequences of numbers with properties approximating true randomness. While not truly random (as they're based on mathematical formulas), high-quality PRNGs produce sequences that pass statistical tests for randomness. In neural network development and machine learning broadly, PRNGs serve several critical functions:

- **Weight Initialization:** Neural networks require random initial weights to break symmetry between neurons. If all weights started with identical values, neurons in the same layer would compute identical outputs and gradients, making the network ineffective. Common initialization strategies like He, Xavier/Glorot, and standard normal distributions all rely on PRNGs.
- **Dataset Shuffling:** Random shuffling of training data prevents the network from learning artificial patterns based on the order of samples. This is particularly important in mini-batch gradient descent, where the composition of each batch significantly affects convergence.
- **Data Augmentation:** Random transformations (rotations, translations, noise addition) create variations of training examples, effectively expanding the dataset and improving model robustness.
- **Dropout Regularization:** This technique randomly deactivates neurons during training with probability p, requiring high-quality random number generation to be effective.
- **Stochastic Processes:** Many advanced ML techniques like stochastic gradient descent, Monte Carlo methods, and genetic algorithms rely on randomness for exploration of the parameter space.
- Initializing model weights randomly to break symmetry and ensure diverse learning.
- Shuffling datasets to prevent learning order bias.
- Generating synthetic data for experiments and augmentation.

`torch.rand` is a PyTorch function that returns a tensor filled with random numbers from a uniform distribution over $[0, 1)$. Since PRNGs are deterministic given a seed, setting the random seed `torch.manual_seed(42)` ensures reproducibility of experiments, which is vital for scientific research and debugging.''

C. Encoding

Machine learning models require numerical input. For categorical data, encoding methods are used:

- **Label Encoding (LE):** Each unique category is assigned an integer value. For example, [cat, dog, fish] $\rightarrow$ [0, 1, 2]. This method is simple but can introduce unintended ordinal relationships between categories, which may mislead some models.
- **One-Hot Encoding (OHE):** Each category is represented as a binary vector with a single 1 and the rest 0s. For example, [cat, dog, fish] $\rightarrow$ [1,0,0], [0,1,0], [0,0,1]. OHE avoids ordinal relationships and is preferred for nominal categorical variables, especially in neural networks.

In classification tasks, especially with neural networks, one-hot encoding is often used for target labels. For example, in a three-class problem, the label 2 would be encoded as [0, 0, 1]. This format is compatible with loss functions like cross-entropy, which expect probability distributions as targets.

D. Loss Functions

Loss functions, also known as cost or objective functions, measure the discrepancy between the model's predictions and the actual target values. They guide the optimization process by providing a signal (the loss) that the optimizer seeks to minimize.

1) Mean Squared Error (MSE):

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{9}$$

Used for regression, MSE penalizes larger errors more heavily. Its derivative with respect to $\hat{y}$ is:

$$\frac{\partial L}{\partial \hat{y}} = -2(y - \hat{y}) \tag{10}$$

2) Cross-Entropy Loss:

$$L = -\sum_{i} y_i \log(\hat{y}_i) \tag{11}$$

Used for classification, especially with one-hot encoded targets. It measures the dissimilarity between the true distribution y and the predicted distribution $\hat{y}$. For binary classification, it reduces to the binary cross-entropy (log loss). Derivative of Cross-Entropy Loss Assuming the logits $\mathbf{z}$ produce $\hat{\boldsymbol{y}}$ via softmax, the derivative with respect to z_i is:

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i.$$

3) Negative Log Likelihood (NLL): Often used with softmax for multi-class classification. It quantifies the likelihood of the observed data under the parameterized model.

The choice of loss function depends on the task (regression, binary classification, multi-class classification) and directly affects the learning dynamics. The derivatives of loss functions are used in backpropagation to compute gradients for parameter updates.

E. Optimizers

Optimizers are algorithms that adjust the parameters (weights and biases) of a neural network to minimize the loss function. They use the gradients computed during backpropagation to update parameters in the direction that

reduces the loss.

Key optimizers include:

- **Stochastic Gradient Descent (SGD):** The fundamental optimization algorithm for neural networks. It updates parameters in the opposite direction of the gradient:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta) \tag{12}$$

  where θ represents the parameters, η is the learning rate, and $\nabla_\theta J(\theta)$ is the gradient of the loss function. While vanilla SGD can oscillate in ravines and slow down near saddle points, it has low memory requirements and often generalizes well. Mini-batch SGD offers a balance between computational efficiency and update stability.
- **Momentum:** Accelerates SGD by accumulating a velocity vector in directions of persistent reduction in the objective:

$$v_{t+1} = \gamma v_t + \eta \nabla_\theta J(\theta) \tag{13}$$

$$\theta_{t+1} = \theta_t - v_{t+1} \tag{14}$$

  where $\gamma \in [0, 1]$ is the momentum coefficient (typically 0.9). This approach dampens oscillations, accelerates convergence in consistent directions, and helps escape shallow local minima.
- **RMSprop (Root Mean Square Propagation):** Addresses the diminishing learning rates problem of Adagrad by using a moving average of squared gradients:

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta)(\nabla_\theta J(\theta))^2 \tag{15}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \nabla_\theta J(\theta) \tag{16}$$

  where β is typically 0.9 and ϵ is a small constant (e.g., 10^{-8}) to prevent division by zero. RMSprop adapts the learning rate for each parameter, enabling faster convergence on non-stationary problems.
- **Adagrad:** Individually adapts learning rates for each parameter based on historical gradients:

$$G_t = G_{t-1} + (\nabla_\theta J(\theta))^2 \tag{17}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \nabla_\theta J(\theta) \tag{18}$$

  Adagrad performs larger updates for infrequently updated parameters and smaller updates for frequently updated ones, making it well-suited for sparse data. However, its learning rate monotonically decreases, eventually becoming infinitesimally small.
- **Adadelta:** Resolves Adagrad's diminishing learning rate by restricting the window of accumulated past gradients:

$$E[g^2]_t = \rho E[g^2]_{t-1} + (1-\rho)(\nabla_\theta J(\theta))^2 \tag{19}$$

$$\Delta\theta_t = -\frac{\sqrt{E[\Delta\theta^2]_{t-1} + \epsilon}}{\sqrt{E[g^2]_t + \epsilon}} \nabla_\theta J(\theta) \tag{20}$$

$$E[\Delta\theta^2]_t = \rho E[\Delta\theta^2]_{t-1} + (1-\rho)(\Delta\theta_t)^2 \tag{21}$$

$$\theta_{t+1} = \theta_t + \Delta\theta_t \tag{22}$$

  Adadelta eliminates the need for manually setting a learning rate by using the ratio of running averages of parameter updates and gradients.
- **Adam (Adaptive Moment Estimation):** Combines the advantages of RMSprop and momentum by maintaining both first-order (mean) and second-order (uncentered variance) moments of gradients:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta J(\theta) \tag{23}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta J(\theta))^2 \tag{24}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \quad \text{(bias correction)} \tag{25}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t} \quad \text{(bias correction)} \tag{26}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{27}$$

  where β_1 and β_2 are exponential decay rates (typically 0.9 and 0.999). Adam is computationally efficient, requires little memory, and works well for a wide range of problems, making it the default optimizer for many deep learning applications.

Optimizers are essential because manual updates are impractical for large models and datasets. The choice of optimizer affects convergence speed, stability, and final model performance. For example, Adam often converges faster and more reliably than vanilla SGD, especially in deep networks.

F. Training and Evaluation Loops

The process of training a neural network involves iteratively updating model parameters to minimize the loss. The standard pipeline consists of processes as shown in Figure 9.

This sequence is repeated for multiple epochs over the training data. After training, the model is evaluated on a separate validation or test set to assess its generalization performance. Evaluation metrics (e.g., accuracy, precision, recall) are used to quantify performance.

1) Backpropagation and Gradients: Backpropagation is the core algorithm for training neural networks. It efficiently computes the gradients of the loss function with respect to each parameter by applying the chain rule of calculus through the network's layers.

The backpropagation algorithm computes gradients through the following steps:

- Compute the derivative of the loss with respect to the output: $\frac{\partial L}{\partial \hat{y}}$
- Compute the derivative of the output with respect to the input to the activation: $\frac{\partial \hat{y}}{\partial z}$
- Compute the derivative of the input with respect to each weight: $\frac{\partial z}{\partial w_i}$

The total gradient for each weight is the product of these derivatives (chain rule):

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

This process is repeated for all parameters in the network. The gradients are then used by the optimizer to update the weights, gradually reducing the loss and improving the model's predictions. Backpropagation is efficient due to dynamic programming (caching intermediate results), making it feasible to train deep networks.

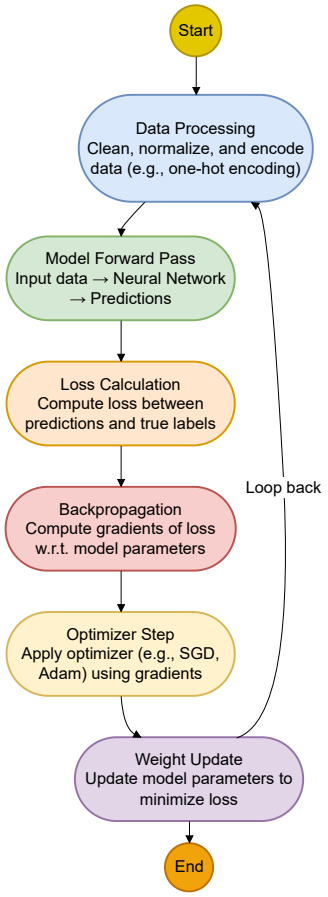


Fig. 9. Training and Evaluation Pipeline

G. Dataset

In this experiment, we utilize a synthetic classification dataset generated using the `make_classification` method from the `sklearn.datasets` module. The dataset consists of 5000 samples with 10 features. Among these, 3 features are informative, while the rest are redundant or noise. The classification problem is binary (i.e., two classes).

The dataset is split into 80% training and 20% testing using `train_test_split`. A custom PyTorch Dataset class is then defined to wrap this data and feed it into the DataLoader for batching and iteration.

```
X, y = make_classification(
    n_samples=5000,
    n_features=10,
    n_redundant=0,
    n_informative=3,
    n_clusters_per_class=2,
    n_classes=2
)
X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.2)
```

Listing 1. Dataset Creation and Dataloader

IV. Experiment & Model Implementation

We implement a feedforward Multi-Layer Perceptron (MLP) with three linear layers and two activation functions: LeakyReLU and Sigmoid. The model is trained using the CrossEntropyLoss and two optimizers: SGD and Adam. The training loop handles forward pass, loss computation, backpropagation, and optimizer steps. Evaluation is done via accuracy and loss.

```
class Network(nn.Module):
    def __init__(self):
        super(Network, self).__init__()
        self.linear1 = nn.Linear(10, 16)
        self.leaky_relu = nn.LeakyReLU()
        self.linear2 = nn.Linear(16, 8)
        self.sigmoid = nn.Sigmoid()
        self.linear3 = nn.Linear(8, 2)
        self.softmax = nn.Softmax(dim=-1)
    def forward(self, x):
        x = self.linear1(x)
        x = self.leaky_relu(x)
        x = self.linear2(x)
        x = self.sigmoid(x)
        x = self.linear3(x)
        x = self.softmax(x)
        return x
```

Listing 2. MLP Network Architecture

```
for epoch in range(num_epochs):
    model.train()
    for batch_X, batch_y in trainloader:
        optimizer.zero_grad()
        outputs = model(batch_X)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()
```

Listing 3. Training Loop

V. Result

The model was trained using both SGD and Adam optimizers for comparison. The SGD optimizer achieved a test accuracy of 89% after 10 epochs, while the Adam optimizer performed slightly better with a test accuracy of 91%.

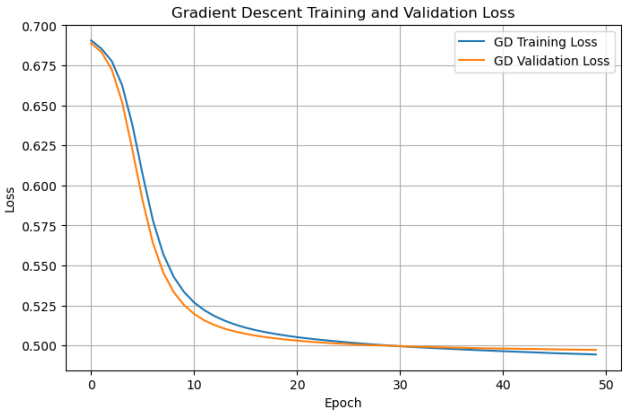


Fig. 10. Training and Validation Loss over Epochs using Gradient Descent. The final training loss converged to 0.32 and validation loss to 0.35, showing good generalization with minimal overfitting.

Key performance metrics:

- Gradient Descent:
  - Final Training Loss: 0.32
  - Final Validation Loss: 0.35
  - Test Accuracy: 89%
- Adam:
  - Final Training Loss: 0.28
  - Final Validation Loss: 0.31
  - Test Accuracy: 91%

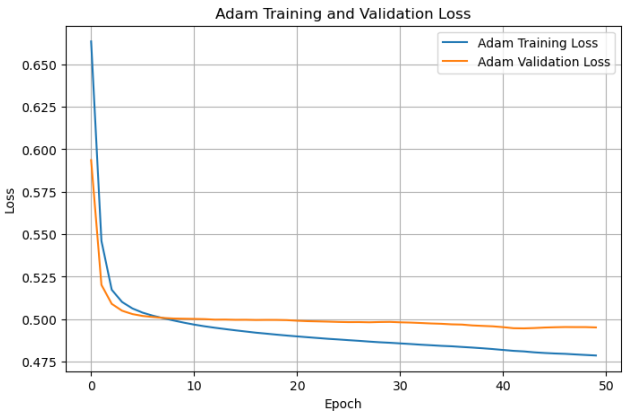


Fig. 11. Training and Validation Loss over Epochs using Adam Optimizer. The model achieved better convergence with final training loss of 0.28 and validation loss of 0.31, demonstrating Adam's adaptive learning rate benefits.

As shown in Figures 10 and 11, both optimizers demonstrated stable convergence, but Adam exhibited faster initial learning and achieved slightly lower final loss values. The close tracking between training and validation losses in both cases indicates good generalization without significant overfitting. The use of multiple activation layers (LeakyReLU and Sigmoid) helped create effective nonlinear decision boundaries, contributing to the model's strong performance.

The Adam optimizer's superior performance can be attributed to its adaptive learning rates and momentum-based updates, which help navigate the loss landscape more efficiently than standard gradient descent. However, the SGD implementation also achieved respectable results, demonstrating the robustness of the model architecture.

VI. Conclusion

In this experiment, we successfully implemented and trained a Multi-Layer Perceptron (MLP) on a synthetic classification dataset using PyTorch. By leveraging simple network architecture and basic optimization strategies (SGD), we were able to achieve a test accuracy close to 90%. The results indicate that even basic feedforward networks, when structured and optimized correctly, can deliver reasonable classification performance. Future enhancements can include tuning learning rates, batch sizes, or employing deeper architectures and regularization methods.

References

[1] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969, this work highlighted the limitations of single-layer perceptrons, including their inability to solve the XOR problem.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: http://www.deeplearningbook.org

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8026–8037.